

WESTERN UNIVERSITY - CANADA
FACULTY OF ENGINEERING
DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING
SE 3314B – COMPUTER NETWORKS APPLICATIONS

Assignment 1: Client/Server Application

Due Date: March 6th, 2026, at 11:59 pm

The evaluation of this assignment is divided as follow: 40% for the implementation (coding) and 60% for the demonstration. The coding component assesses correctness, protocol design, structure, and adherence to the specifications. The demonstration assesses your understanding of the system, your ability to clearly explain design decisions, and the correct live operation of your implementation. An incomplete code cannot achieve a perfect demo mark.

AI tools may be used to assist with implementation (including generating code), but the design must be your own. You are expected to first attempt the problem independently, identify the key challenges, and decide on the protocol behavior, state management, and overall architecture before using AI. You are fully responsible for all submitted code and must understand, validate, and be able to explain every design choice and every part of

1 Objectives

This assignment is a refresher on socket programming using Node.js. This assignment is a refresher for socket programming. I assume you have done some socket programming prior to this course, either in ECE4436a or equivalent experience. If your socket programming skill is rusty or shaky or if you're still unsure how to build a socket program using the Node.js environment, this assignment will give you a chance to review and improve both. You are provided with a JavaScript code skeleton and supporting methods to build a client-server multimedia viewer. The server will respond to client requests by transferring image or video files, which the client will reconstruct and display using the system's default media viewer. A correct implementation will successfully render recognizable media, providing immediate feedback on correctness.

In addition to standard file transfer, this assignment introduces a secret sharing extension. Clients may initiate a special session in which the server provides a riddle requiring specific file requests in a defined order. When the correct sequence is followed, the server distributes portions of a secret key across multiple responses. Once all key parts are collected, the client can request and decrypt a protected secret file. This extension requires careful protocol design, state tracking, and validation beyond basic socket communication.

The completed system will be used for assignment 2 and implementing with peer-to-peer networking concepts.

2 Installing Node.js

The first thing to do before you begin is install the JavaScript compiler (the Node.js platform). If you have done so already, please skip this section. Node can be downloaded from the project home page: <http://nodejs.org/> where you will be presented with a package installer for your operating system. Binary installers are available for Unix, Mac, and Windows. The stable release version of Node at the time of this assignment is 24.13.1 (LTS). Node provides a JavaScript runtime environment, which you can access at any time by going into your command prompt or terminal window and typing `node`. For the purposes of the assignment, you will be running your main program (JavaScript source file) rather than typing code directly into Node. To run a JavaScript file, you run the `node` command with a parameter like this: `node filename.js`.

3 Importing third-party modules

Node ships with a **package management utility** called **npm**, which enables you to import third-party libraries into your workspace to use in your code. Package management is an important aspect of working with Node; without it, you would have to program all of your applications from the ground up, reinventing solutions to common problems that have already been solved (and shared) by hundreds of other developers.

To install these third-party modules (like `open` package) using `npm`, run the `npm` command with the module name as its parameter, like this: `npm install open --save`.

As you develop Node.js applications, you will undoubtedly make use of modules. In order to install all your application modules automatically when being deployed in the running and evaluation device, you need to use the `package.json` file concept.

4 Task 1: The Media Management Server

You are required to develop an asynchronous media management server that can handle client's requests individually and service many (theoretically hundreds or thousands) clients at the same time.

This means, as each client connects to the server port and host, the main program will fire a pre-defined **Callback** function asynchronously. Because the `server.listen()` method creates a new **Socket** object for the connection, the original Socket object will be free to accept more client connections. On the command line, the server is run simply by typing:

```
> node MediaDB
```

The server does not have any required command-line option. Upon start up, the server initializes a listening socket and obtains an ephemeral port number from the operating system, which it prints out to the console. The server then waits for requests from clients, loads the requested media from the set of available files in the same folder or directory it is launched from, and transfers the media to the client using the defined MTP protocol (defined below).

In addition to standard media queries, the server must support a secret session mode. When a client initiates a secret session, the server returns a riddle that encodes a required sequence of file requests. If the client requests the correct files in the correct order, the server distributes the secret key in three parts. Key distribution is performed across multiple responses using the

```

○ node MediaDB

Server listening on 127.0.0.1:3000
Connected: 127.0.0.1:62774
Client joined: 127.0.0.1:62774

Client-1476 is connected at timestamp: 1476

MTP packet received:
10010000 00000000 00000000 00000000
00000000 00000000 00000010 01000000
00010000 00000000 00000000 00001000
01000011 01100001 01110010 01100100
01101001 01101110 01100001 01101100

Client-1476 requests:
--MTP version: 18
--Timestamp: 576
--Request type: Query
--Image file extension(s): JPEG
--Image file name: Cardinal

```

Figure 1: Sample look of your server interface (Incorrect version)

header defined in the next section. Once all key parts are received, the client can request the secret file and decrypt it locally using the reconstructed key. To ensure the secret varies across connections and is only valid within a time frame, the server selects among three secret variants using a time-based rotation. The server defines a fixed rotation window (for example, 300 seconds). Each secret session is associated with the current window and becomes invalid once the window changes. If the client attempts to continue a secret exchange after a window change,

the server must treat the exchange as failed and require the client to reset and restart the secret process. You are responsible to design how this component will work. You must also generate the key, secret and riddle (using the names of the available files).

The secret session should use a cipher such as [Vigenère cipher, The Black Chamber - Vigenère Cipher .](#) Vigenere a classic cipher used for encrypting text. It is okay to use other ciphers as desired, only if they are more complex than Vigenere. The key length must at least be 10 ASCII characters long. The secret file can be a text file, and the original text file should contain at least 40 ASCII characters long. You would need three different keys and three different secret files for the secret rotation (i.e.- the same text can be encrypted with different keys). There are online tools that may help with ciphering and deciphering (<https://www.dcode.fr/vigenere-cipher>). When sending the key in three parts, you would have to decide the appropriate chunk size per each part, but it must be possible to reconstruct 1/3 (max) of the original key with a single part.

The secret sharing exchange requires an acknowledgment only when a key part is sent. When the server sends a response that contains a key part, the client must explicitly acknowledge receipt of that key part. The server must not advance the secret session state until the correct acknowledgment is received. For normal file transfers that do not include a key part, no acknowledgment is required beyond the standard request-response behavior.

Your software application should include, but is not limited to, the following components:

4.1 Console Interface

The Node.js command console will be an acceptable UI for your application in this assignment. Your application will display the core operations of the server sequentially and allow interaction through scrolling. Figure 1 shows a sample look of your server interface.

4.2 Response Packetization

Module – MTPResponse.js

The media management server uses an SE3314b' custom Media Transport Protocol (MTP) for media files delivery. The core server functionality is to maintain the client's connection, send

and receive the MTP packets (i.e., the encapsulated MTP packets) and provide the secret key. However, the main functionality of this module is to maintain the MTP packetization process, i.e., to form the MTP response packet. The size of MTP response packet **header** is 12 and only 12 bytes and should be sent in specific **bits** order as shown in Figure 2. It is recommended to use the additional methods given in the provided code to form the header bit stream.

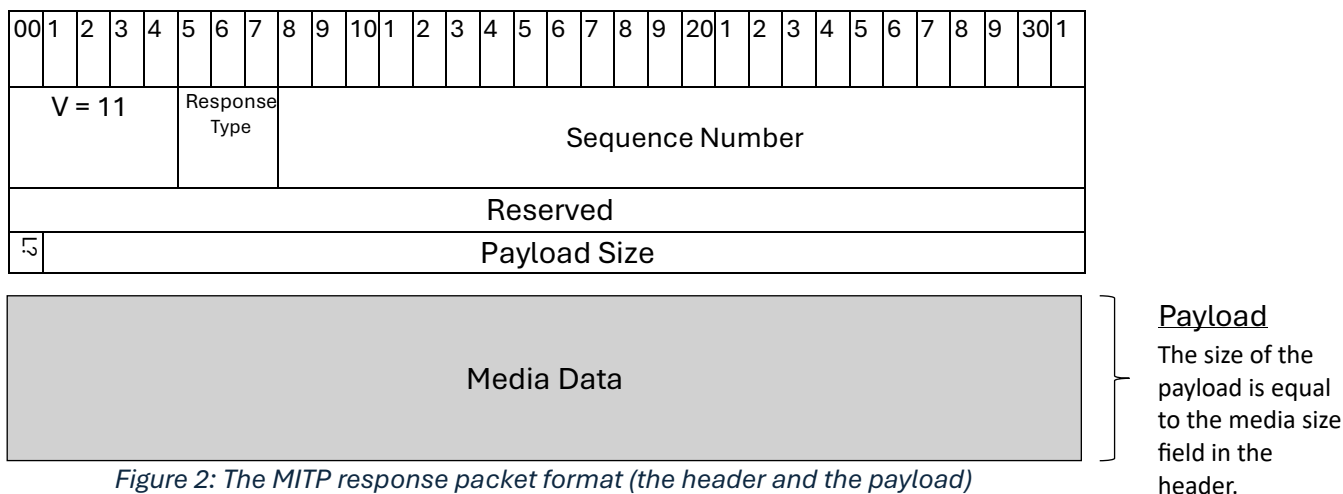


Figure 2: The MITP response packet format (the header and the payload)

Here are the descriptions of the MTP response packet fields:

- (V) is a 5-bit MTP version field. You must set this to 11. The bits order should be as follows:

Bit Position	0	1	2	3	4
Bit Value	0	1	0	1	1

- The response type is a 3-bit value so that the value 0 means “Query”, 1 means “Found”, 2 means “Not found”, and 3 means “Busy”. Other values may be used later.
- Set the sequence number. The sequence number of the first packet should start at 0; it is incremented by 1 for each subsequent packet sent out of the server. This field is 24 bits in length and the increment will be performed mod 2^{24} .
- The reserved field is a 32-bit field will be used for secret session purposes.
 - For normal media responses, this field may be set to 0.
 - For secret sessions, this field is used to carry secret-session control information such as the current time window identifier and, when applicable, a key part (or key part identifier plus key part data, depending on your encoding design).
- The L? is a Boolean flag (1bit) that should indicate whether this is the last packet for the requested file.

- The payload size is a 31-bit field which we will need to extract the payload data from the packet and display it.
- The actual payload data (part of the payload) will then follow.

Note that depending on the type of data you are sending, you may have to send multiple packets per request. You should use the sequence number and the L? Flag to your advantage.

4.3 Clients Handler Module – ClientsHandler.js

This module is responsible for accepting incoming client connections, receiving request packets, validating and parsing request headers, and sending the appropriate response packets.

The protocol must support the following request message types:

- Query – Standard media file request
- Secret – Initiates a secret session
- Reset – Resets an existing secret session
- ACK – Acknowledges receipt of a key part
- Complete – Requests the final secret file

When a request is received, the module must:

- Validate that the header is well-formed and that the version field matches the expected protocol version.
- Identify the request type.
- Route the request to the appropriate handling logic.

For a **Query** request, the server must determine whether the requested media exists and responds accordingly, using single or multiple packets as needed. If none of the requested media exists, it sends only the completed MTP packet (no payload) with response type “Not found”. If the media files (or some of them) exists, it sends the MTP packets and the payload(s) to the client. You must fill in all the contents of the packet.

For a **Secret** request, the server must initiate a secret session and respond with the appropriate riddle based on the current secret rotation.

For a **Reset** request, the server must clear any existing secret session state for that client.

For an **ACK** request, the server must process acknowledgment of a previously sent key part before progressing the secret exchange.

For a **Complete** request, the server must verify whether the secret exchange has been successfully completed before sending the final protected file.

The module must maintain per-client state across connections for secret sessions. Normal media queries do not require persistent state.

4.4 Main Application – MediaDB.js

The main application of the server requires the above modules (MTPpacket.js, ClientsHandler.js, and other like fs, etc.) in order to function. The basic functionality of the main application is to accept connections, receive client command and respond accordingly.

It is also responsible to display (log) into the console all the information about the connected/disconnected clients, and client's requests (MTP request packet bits stream along with the data fields) as shown in Figure 1.

5 Task 2: The Client Side

Your second task is to write the client code. The client takes a single line command with the following format:

```
> node GetMedia -s <serverIP>:<port> -q <media name> -v <version>  
--type <secret/query>
```

Where '>' indicates the console prompt, which you don't type in. The -r option tells **GetMedia** which server to connect to. You must provide the IP address of the server, along with the port

number the server is listening on, separated by a colon. The angle brackets (" $<$ " " $>$ ") indicate that you are to provide the actual values for the required arguments. You don't enter the angle brackets themselves when you run the program. The `-q` option tells the **GetMedia** program which media files to query the server for. You replace " $<$ media name" with the media file name including the media type, again you don't enter the angle brackets themselves. The `-v` option allows you to change the version number of the MTP packet sent. The `--type` option specifies whether the request should be handled in secret mode or as a standard query request. You should use it to test whether your server function is checking the version field of all incoming MTP packets correctly. In this assignment, 11 is the supported version of the MTP.

Your software application in this task **should include, but is not limited to**, the following components.

5.1 Console Interface

Your application will display the core operations of the client side sequentially and use a scrolling up form of interaction. Figure 3 shows a sample look of your client interface.

```
node GetMedia -s 127.0.0.1:3000 -q Cardinal.jpeg -v 18
Connected to Images server on: 127.0.0.1:3000
Received packet bits:
10010001 00000000 00000000 00000000
00000000 00000000 01101011 10010110
00000000 00000000 00111011 01000001

Version: 18
Response type: Found
Current time: 27542
Received packet bits:
Processing complete packet 0, payload size: 15169

Received packet bits:
10010001 00000000 00000000 00000001
00000000 00000000 01101011 10010110
10000000 00000000 00000000 00000000

Version: 18
Response type: Found
Current time: 27542
Received packet bits:
Processing complete packet 1, payload size: 0

File saved successfully with size: 15169
Total packets received: 2

Disconnected from the server
Connection closed
```

Figure 3: Sample look of your client interface (wrong version)

5.2 Request packetization Module – MTPRequest.js

As you already know, our media management server uses an SE3314b custom Media Transport Protocol (MTP) for media files delivery. The main functionality of this module is to

maintain the MTP packetization process, i.e., to form the MTP request packet. The MTP request packet is a 12 bytes header followed by the payload (the media file name). The MTP request packet should be sent in a specific structure as shown in Figure 4.

It is recommended to use the additional methods given in the provided code to form the header bit stream.

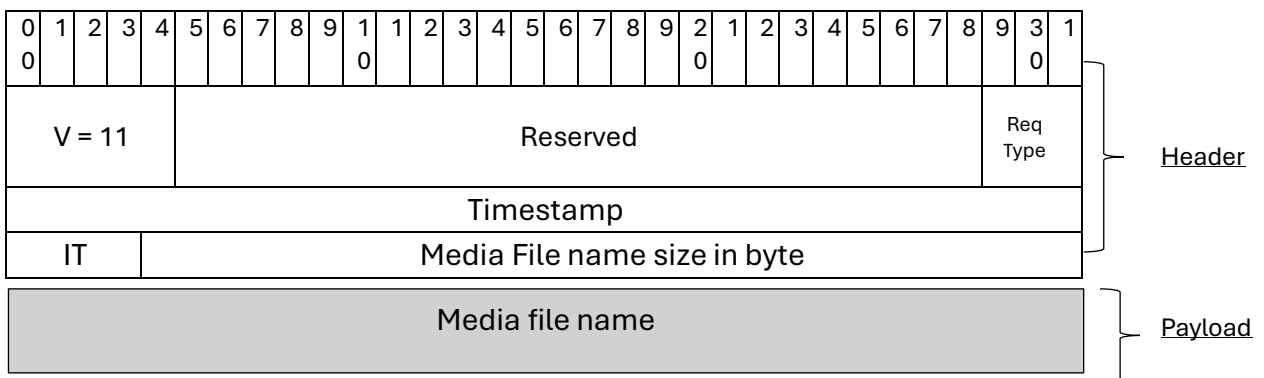


Figure 4: The MTP request packet format

Here are the descriptions of the MTP request packet fields:

- (V) is a 5-bit MTP version field. You must set this to 11.
- Reserved is a 24-bit field, not used in this assignment.
- The request type is a 3-bit field. In this assignment value 1 means "Query", 2 means "Secret", 3 means "ACK", 4 means "Complete" and 5 means "Reset".
- The timestamp is a 32-bit field that has the current value of the client's timer.
- (IT) Media Type is a 4-bit field that indicates the type of the media as follows: 1 – BMP, 2- JPEG, 3- TIFF, 4- GIF, 5- PNG, 15- RAW, 6-AVI, 7- MP4, 8- MOV
- The file name size is a 28-bit field that holds the number of bytes needed to store the media file name.
- The media file name, **without the file extension**, is a field to store the name of the requested media. The size of this field is equal to the length of the media file name.

5.3 Main Application – GetMedia.js

This module provides the client-side networking logic. It is responsible for connecting to the server using the IP address and port provided in the command-line arguments, sending a properly formatted request packet, and receiving and processing the server's response packets.

The first task of this module is to connect to the server using the IPv4 address and port number given in the command line arguments. At the same time as the client connects to the server, it sends a query packet, which includes the required media file name, to the server using the packet structure described in Figure 4. This module will also receive the return packet from the server. The return packet structure must be of the form shown in Figure 2 above. Once this module receives all the correct MTP packet format, it extracts the media contents from the packets and saves it in the client's current folder. Finally, it calls the default media viewer in the client's computer to display the downloaded media. You may need to use the third-party NPM package called 'open'.

When the client is run in secret mode, it must support the secret session workflow. This includes initiating a Secret request, issuing the required follow-up requests, sending an ACK message when a key part is received, issuing Reset if the secret exchange fails, and sending a Complete request once the required key parts have been collected. When the Complete request is sent and the server responds with the secret file, the client must reconstruct the received file, save it locally, and open it using the default viewer.

6 Testing Your Code

You can develop your code on either Linux, Mac OS X, or Windows platform, as Node.js is available for all these platforms and the code you develop on one platform can run directly on the others. The easiest way to test your code is to run both the server and client on your local host, e.g., your laptop. After completing the code as described above, you can run it as follows:

- First, start the server as shown below and recognize the port number and server address on which the server is accepting client connections.

```
> node MediaDB  
  
MediaDB server is started at timestamp: 265 and is listening on 127.0.0.1:3000
```

- Then start the client program 'GetMedia', as shown below. Enter the recognized server port number and server address, include media file names you need to request. Note that there is flexibility on what this can look like, as long as you meet the requirements.

```
> node GetMedia -s 127.0.0.1:3000 -q Rose.gif -v 11
Connected to MediaDB server on: 127.0.0.1:3000

MTP packet header received:
10010100 00000000 00000000 11110111
00000000 00000000 00000011 10100011
00000000 00000000 11100001 00010101

Server sent:
  --MTP version = 11
  --Response Type = Found
  --Sequence Number = 0

Disconnected from the server
Connection closed
```

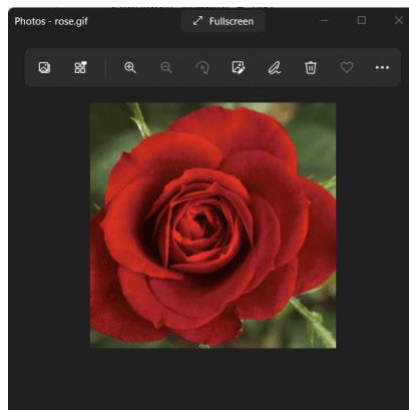


Figure 5

- The server should accept the client's request and update its screen as shown below. Note that, the server is always running and accepting requests from many clients.

```
> node MediaDB
MediaDB server is started at timestamp: 265 and is listening on: 127.0.0.1:3000

Client-931 is connected at timestamp: 931

MTP packet received:
10010000 00000000 00000000 00000000
00000000 00000000 00000011 01000001
01010000 00000000 00000000 00000100
01010010 01101111 01110011 01100101

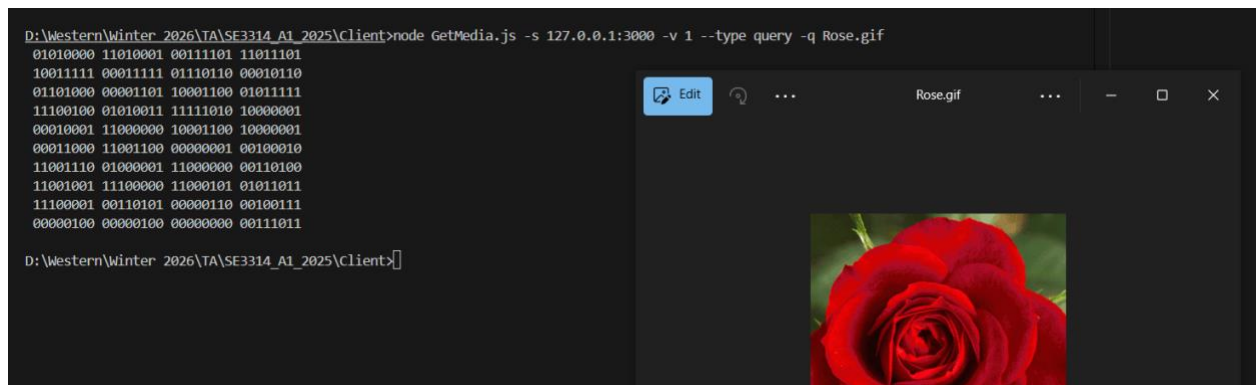
Client-931 requests:
--MTP version: 11
--Request type: Query
--Media file extension(s): GIF
--Media file name: Rose

Client-931 closed the connection
```

Secret Session Example:

New Images (key = plzunlckandsee)

1. Normal query

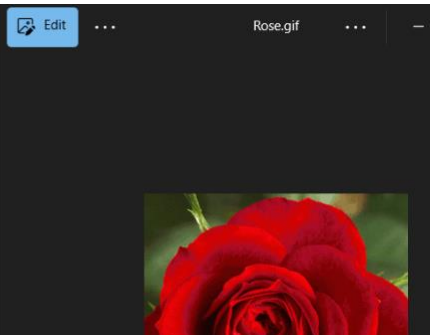


2. Starting a secret session



3. Sending the files in order (e.g., the first one)

```
D:\Western\Winter 2026\TA\SE3314_A1_2025\Client>node GetMedia.js -s 127.0.0.1:3000 -v 1 -q Rose.gif
01010000 11010001 00111101 11011101
10011111 00011111 01110110 00010110
01101000 00001101 10001100 01011111
11100100 01010011 11111010 10000001
00010001 11000000 10001100 10000001
00011000 11001100 00000001 00100010
11001110 01000001 11000000 00110100
11001001 11100000 11000101 01011011
11100001 00110101 00000110 00100111
00000100 00000100 00000000 00111011
01000001 01000011 01001011 00100000
01110010 01100101 01100011 01100101
01101001 01110110 01100101 01100100
00101110
ACK received.
```



4. A key part is received (during the sequence)

```
D:\Western\Winter 2026\TA\SE3314_A1_2025\Client>node GetMedia.js -s 127.0.0.1:3000 -v 1 -q bunny.mp4
00101011 01101001 01101100 01110011
01110100 00000000 00000000 00000000
00100011 10101001 01110100 01101111
01101111 00000000 00000000 00000000
00011011 01100100 01100001 01110100
01100001 00000000 00000000 00000000
00000001 00000000 00000000 00000000
00000000 01001100 01100001 01110110
01100110 00110101 00110011 00101110
00110010 00110100 00101110 00110010
Key part received: ckand
01000001 01000011 01001011 00100000
01110010 01100101 01100011 01100101
01101001 01110110 01100101 01100100
00101110
ACK received.
```

5. An error (not completing the sequence)

```
D:\Western\Winter 2026\TA\SE3314_A1_2025\Client>node GetMedia.js -s 127.0.0.1:3000 -v 1 -q 4u.txt
01000101 00100000 01101011 01100101
01111001 00100000 01100101 01111000
01100011 01101000 01100001 01101110
01100111 01100101 00100000 01100010
01100101 01100110 01101111 01110010
01100101 00100000 01110010 01100101
01110001 01110101 01100101 01110011
01110100 01101001 01101110 01100111
00100000 00110100 01110101 00101110
01110100 01111000 01110100 00101110
COMPLETE key exchange before requesting 4u.txt.
```

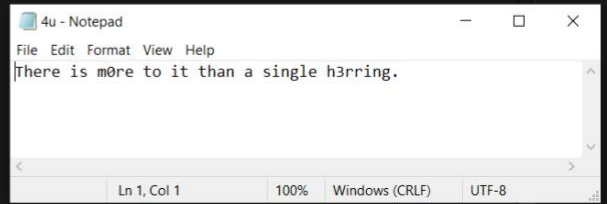
6. Final key part received, and when requested for the text, it is sent by the server (without error) and can be deciphered now.

```

D:\Western\Winter 2026\TA\SE3314_A1_2025\Client>node GetMedia.js -s 127.0.0.1:3000 -v 1 -q planet.mov
00001010 00100011 01001110 00000000
00001010 00100110 01010101 00000000
00000000 00000000 00100001 01110101
01100100 01110100 01100001 00000000
00000000 00000000 00011001 10101001
01110011 01110111 01110010 00000000
00001101 01010101 11000100 01001100
01100001 01110110 01100110 00110101
00101111 00101110 00110001 00111001
00101110 00110001 00110000 00110000
Key part received: see
01000001 01000011 01001011 00100000
01110010 01100101 01100011 01100101
01101001 01110110 01100101 01100100
00101110
ACK received.
01000011 01001111 01001101 01010000
01001100 01000101 01010100 01000101
00100000 01100001 01100011 01100011
01100101 01110000 01110100 01100101
01100100 00101110
COMPLETE accepted.

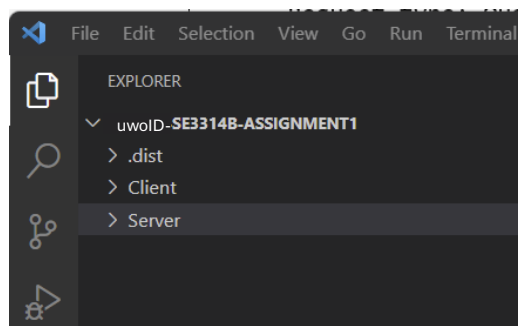
D:\Western\Winter 2026\TA\SE3314_A1_2025\Client>node GetMedia.js -s 127.0.0.1:3000 -v 1 -q 4u.txt
01100100 01101100 01110010 00100000
01110100 01110101 00100000 01110111
00110000 01110010 01110010 00100000
01110111 01100111 00100000 01101101
01111000 00100000 01101001 01110011
01111010 01101000 00100000 01101110
00100000 01100100 01101011 01111000
01100111 01111001 01101000 00100000
01111010 00110011 01110110 01110110
01111000 01111001 01100110 00101110

```



7 Submission

- Your source code should be fully commented and formatted.
- Your project folder structure should be created such that all the code for the server modules should be under a separate folder named 'Server' and all the code related to the client program should be saved in a folder named 'Client', as follows (replace uwold appropriately):



- Compress these two folders in an archive file (zip file) and name it **yourUWOLD-SE3314b-assignment1.zip**. This should include all the JS files for the server and the client including the **package.json** file.
- Submit this zip file on OWL by the due date mentioned above.

- Information regarding the Demo and marking scheme will be communicated in a separate file.